

SESSION 1: MULTI-SCALE PARALLELISM STRATEGIES

Data Parallelism Fundamentals

Distributing Datasets for Parallel Training

Core Concept:

Data parallelism splits training datasets across multiple devices, with each device maintaining a complete copy of the model. This approach scales naturally with dataset size and provides linear speedup under ideal conditions.

Mathematical Foundation:

For N devices processing batch size B :

- Global gradient equals to the average of local gradients: $g_{global} = (1/N) \times \sum(g_i)$ where g_i is local gradient
- Equivalent to single device with batch size $N \times B$
- Maintains SGD (Stochastic Gradient Descent) mathematical properties

Device Coordination:

1. Dataset splitting into non-overlapping subsets
2. Independent forward and backward passes
3. Gradient synchronization via ring all-reduce
4. Synchronized parameter updates

Scaling Benefits:

Linear speedup possible up to communication bottleneck threshold.

Quick Review – Stochastic Gradient Descent

The core idea of SGD is to **update the model's parameters** using the **gradient** calculated from a **small, randomly selected subset of the training data**, known as a mini-batch, rather than the entire dataset. In its purest form, SGD updates parameters using only a single data point at a time, though using mini-batches is more common in practice and often referred to as SGD.

- **Mathematical Formulation:** For model parameters ϑ , learning rate η , and loss $L(\vartheta)$:
- **Standard Gradient Descent:** $\vartheta_{t+1} = \vartheta_t - \eta \nabla L(\vartheta_t)$ (uses full dataset)
- **Stochastic Gradient Descent:** $\vartheta_{t+1} = \vartheta_t - \eta \nabla L_i(\vartheta_t)$ (uses single random example i)
- **Mini-batch SGD (most common):** $\vartheta_{t+1} = \vartheta_t - \eta \nabla LB(\vartheta_t)$ - Where B is a mini-batch of examples and $\nabla LB(\vartheta_t) = (1/|B|) \sum_{i \in B} \nabla L_i(\vartheta_t)$

Algorithm Steps

1. Initialize model parameters ϑ_0 randomly
2. Shuffle the training dataset
3. For each epoch (pass through the dataset):
 - For each mini-batch B in dataset:
 - Compute gradient $\nabla LB(\vartheta_t)$
 - Update parameters: $\vartheta_{t+1} = \vartheta_t - \eta \nabla LB(\vartheta_t)$
4. Repeat for multiple epochs until the loss becomes sufficiently small or stops improving

Example Update Rule: For linear regression with MSE loss: $L(\vartheta) = (1/2n) \sum (y_i - \vartheta^T x_i)^2$

SGD update for parameter ϑ_j : $\vartheta_j \leftarrow \vartheta_j + \eta (y_i - \vartheta^T x_i) x_{ij}$

Data Parallelism Mechanics

Data Parallelism Fundamentals

Step-by-Step Execution Process

Execution Flow:

Input Data → Split into Batches → Distribute to GPUs



GPU 1: Forward Pass → Backward Pass → Compute Gradients

GPU 2: Forward Pass → Backward Pass → Compute Gradients

GPU 3: Forward Pass → Backward Pass → Compute Gradients

GPU 4: Forward Pass → Backward Pass → Compute Gradients



Gradient Synchronization (Ring All-Reduce)



Parameter Update on All Devices → Next Iteration

Communication Pattern:

Ring all-reduce algorithm minimizes communication overhead:

- Each device communicates only with neighbors
- 7 steps for 8 devices vs. 56 for naive peer-to-peer
- Bandwidth usage scales optimally with device count

Implementation Example:

Modern frameworks like PyTorch DistributedDataParallel handle synchronization automatically.

Data Parallelism Mechanics

Scaling Limitations and Communication Overhead

Communication Bottleneck:

For 1B parameter model with 8 GPUs:

- Gradient data: ~4GB per synchronization step
- High-speed interconnects required (NVLink: 900 GB/s, InfiniBand: 400 Gb/s)
- Communication time grows with model size and device count

Scaling Efficiency:

- 8 GPUs: typically 87.5% efficiency (7x speedup)
- 64 GPUs: typically 70-78% efficiency (45-50x speedup)
- Diminishing returns due to synchronization overhead

Memory Constraints:

175B parameter model requires:

350GB for FP32 parameters

Additional memory for optimizer states and activations

Total: >1TB per device requirement

Exceeds current GPU memory (80GB high-end)

Workload Imbalance:

Heterogeneous systems suffer from device speed mismatches.

Scalability Analysis

Sources of scaling loss

- Communication overhead: bandwidth limits, latency, protocol inefficiency
- Synchronization barriers and stragglers: everyone waits for the slowest
- Load imbalance/heterogeneity: uneven work, throttling, ECC, I/O skew
- Memory pressure: activations/optimizer state constrain batch/parallelism

Measuring efficiency

- Ideal speedup = N
- Actual speedup = T_1 / T_N
- Parallel efficiency = $(T_1 / T_N) / N$

Amdahl's Law (upper bound on speedup)

- Speedup $\leq 1 / (s + (1 - s)/N)$, where s = serial fraction
- Example: $s = 0.10 \Rightarrow \text{max speedup} = 10\times \text{ as } N \rightarrow \infty$
- Even small s severely caps efficiency at large N

What to do (reduce s , hide comm)

- Overlap communication with compute: gradient bucketing, fused kernels
- Hierarchical collectives; topology-aware placement
- Lower-precision or compressed gradients with error feedback
- Algorithmic options: increase local batch; partial/delayed sync; asynchronous variants with care
- Balance work: uniform microbatching, stage rebalancing, dynamic batch assignment
- Detect and mitigate stragglers: health checks, retry, prefetch, pinned workers